



OASIS Security Report

Cross-Site Request Forgery Security Analysis

Date: 2026-06-11 23:33:55 — **Model:** llama3.2:3b

Vulnerability: Cross-Site Request Forgery

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **9** file(s) for Cross-Site Request Forgery.

[Return to table of contents](#)

Statistics

Files analyzed: 9 **Total findings:** 15

Critical



7

High		3
Medium		4
Low		1

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/vulnerable/access-control/mycontract.sol	0.658
/work/project/dataset/vulnerable/reentrancy/etherstore.sol	0.643
/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol	0.637
/work/project/dataset/vulnerable/reentrancy/simple_dao.sol	0.635
/work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol	0.632
/work/project/dataset/vulnerable/ordering-attacks/ eth_tx_order_dependence_minimal.sol	0.617
/work/project/dataset/vulnerable/ordering-attacks/odds_and_evens.sol	0.613
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_1.sol	0.612
/work/project/dataset/vulnerable/access-control/parity_wallet_bug_2.sol	0.596

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.658

Source lines: 1-31

► Finding 1.1: Missing CSRF Token (Critical)

Vulnerable code

```
function sendTo(address receiver, uint amount) public require(tx.origin ==  
owner);  
receiver.transfer(amount);
```

Explanation

The function does not include a CSRF token, making it vulnerable to Cross-Site Request Forgery attacks.

Impact

Unauthorized actions can be performed on behalf of authenticated users.

Entry point

/transfer

HTTP methods

POST

Parameters

tx.origin, amount

Exploitation steps

- Send a malicious tx.origin request to the contract, tricking it into executing the transfer function.

Conditions

Authenticated user with tx.origin control

Remediation

Include a CSRF token in the transfer function and validate its presence.

Secure example

```
require(tx.origin == owner && hasCSRFToken());  
receiver.transfer(amount);  
function hasCSRFToken() public view returns (bool) { return msg.sender !=  
address(0); }
```

File 2: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.643

Source lines: 1-39

► Finding 2.1: Cross-Site Request Forgery (Critical)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw))()
```

Explanation

The call() function executes a function with no return value, making it vulnerable to Reentrancy attacks.

Impact

Unauthorized actions can be performed on behalf of authenticated users

Entry point

/withdrawFunds

HTTP methods

POST

Parameters

_weiToWithdraw, msg.sender

Exploitation steps

- Send a malicious request to the withdrawFunds function with a forged sender address.
- Exploit the call() function's lack of return value.

HTTP raw requests (Burp / ZAP)

```
POST /withdrawFunds HTTP/1.1\r\nHost: example.com\r\nContent-Type: application/json\r\n\r\nmalicious payload\r\n\r\n
```

File 3: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.637

Source lines: 1-85

► Finding 3.1: Vulnerable Code (Critical)

Vulnerable code

```
(function() { msg.sender.call.value(100)(); })().call()
```

Explanation

The `CashOut` function calls `msg.sender.call.value(\_am)` which allows an attacker to trick the user into executing arbitrary code on their behalf.

Impact

Unauthorized actions performed on behalf of authenticated users

Entry point

/transfer

HTTP methods

POST

Parameters

_am

Exploitation steps

- 1. Send a malicious request to /cashout with a large amount (_am).

Example payloads

- ```
POST /cashout HTTP/1.1
Host: example.com
Content-Length: 0

100 ether
```

#### Conditions

Authenticated user

#### Remediation

Verify request origins and validate user input.

## Secure example

```
function CashOut(uint _am) public payable { balances[msg.sender] -= _am; }
```

## File 4: /work/project/dataset/vulnerable/reentrancy/simple\_dao.sol

Similarity score: 0.635

Source lines: 1-35

### ► Finding 4.1: Missing CSRF Token (Critical)

#### Vulnerable code

```
bool res = msg.sender.call.value(amount)();
```

#### Explanation

The function calls msg.sender's `call()` method without verifying a CSRF token, making it vulnerable to Cross-Site Request Forgery attacks.

#### Impact

Unauthorized actions can be performed on behalf of authenticated users.

#### Entry point

/withdraw(uint amount)

#### HTTP methods

POST

#### Exploitation steps

- Attacker sends a POST request to /withdraw with a malicious amount
- The attacker's request is accepted, and the malicious amount is executed

#### Conditions

Authenticated users can reach the /withdraw endpoint.

#### Remediation

Add a CSRF token verification in the `withdraw` function.

## File 5: /work/project/dataset/vulnerable/ordering-attacks/FindThisHash.sol

Similarity score: 0.632

Source lines: 1-28

### ► Finding 5.1: Vulnerability found (Critical)

#### Vulnerable code

```
function solve(string solution) public { require(hash == sha3(solution));
msg.sender.transfer(1000 ether); }
```

#### Explanation

This contract lacks a CSRF token, allowing an attacker to transfer Ether from the contract on behalf of the user.

#### Impact

Unauthorized actions performed on behalf of authenticated users can lead to financial losses or compromise the account.

#### Entry point

/solve

#### HTTP methods

POST

#### Exploitation steps

- Attacker sends a POST request to /solve with a crafted solution string.

#### Example payloads

- `a' transfer 1000 ether`

#### Conditions

User is authenticated and authorized to execute the solve function.

#### Remediation

Implement a CSRF token mechanism, such as reCAPTCHA or a library like CSRF-Fortify.

## File 6: /work/project/dataset/vulnerable/ordering-attacks/eth\_tx\_order\_dependence\_minimal.sol

Similarity score: 0.617

Source lines: 1-42

### ► Finding 6.1: Missing CSRF Token (Critical)

#### Vulnerable code

```
require (!claimed);

require(msg.sender == owner);
```

#### Explanation

The setReward() function does not include a CSRF token, making it vulnerable to Cross-Site Request Forgery attacks.

#### Impact

An attacker could force the owner to transfer funds without their knowledge or consent.

#### Entry point

setReward()

#### HTTP methods

POST

#### Exploitation steps

- An attacker sends a malicious request to the setReward() function with a fake owner address.

#### Conditions

The owner is authenticated and has not claimed the reward yet.

#### Remediation

Add a CSRF token to the setReward() function



## Remediation

Validate user input and ensure proper authorization checks are in place.

### Secure example

```
pragma solidity ^0.8.0;
contract OddsAndEvens {
 function play(uint number) payable{
 require(msg.value == 1 ether, "Invalid value");
 // ...
 }
}
```

## File 8: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_1.sol

Similarity score: 0.612

Source lines: 1-281

### ► Finding 8.1: Vulnerable Code (Medium)

#### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
 if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
 // yes - just execute the call.
 address created;
 if (_to == 0) {
 created = create(_value, _data);
 } else {
 if (!_to.call.value(_value)(_data))
 throw;
 }
 SingleTransact(msg.sender, _value, _to, _data, created);
 } else {
 // determine our operation hash.
 o_hash = sha3(msg.data, block.number);
 // store if it's new
 if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
 m_txs[o_hash].to = _to;
 m_txs[o_hash].value = _value;
 m_txs[o_hash].data = _data;
 }
 if (!confirm(o_hash)) {
 ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
 }
 }
}
```

#### Explanation

The function does not verify the request origin before executing a transaction.

#### Impact

Unauthorized transactions can be executed without user consent.

#### Entry point

/execute(address \_to, uint \_value, bytes \_data)

#### HTTP methods

POST

## Parameters

\_to, \_value, \_data

## Exploitation steps

- An attacker requests a transaction for an invalid address
- The system executes the transaction without verifying the request origin

## Example payloads

- ```
{
  _to: 'Invalid Address',
  _value: 1,
  _data: ''
}
```
- ```
{
 _to: '0x12345678901234567890123456789012345678901234567890',
 _value: 10,
 _data: ''
}
```

## Conditions

The attacker can request a transaction with an invalid address.

## Remediation

Verify the request origin before executing a transaction.

**Source lines:** 27–358

### ► Finding 8.1: Vulnerable event call without CSRF token (High)

#### Vulnerable code

```
event SingleTransact(address owner, uint value, address to, bytes data,
address created);
```

#### Explanation

The `SingleTransact` event call does not include a CSRF token, making it vulnerable to Cross-Site Request Forgery attacks.

#### Impact

An attacker could trick an authorized user into executing unintended transactions on their behalf.

#### Entry point

/execute(address \_to, uint \_value, bytes \_data)

#### HTTP methods

POST

## Parameters

\_to, \_value, \_data

## Exploitation steps

- An attacker sends a malicious SingleTransact event call to the contract.
- The contract executes the transaction without verification, allowing the attacker to control the outcome.

## Example payloads

- malicious payload data

## Conditions

Attacker has access to authorized user's credentials or can trick them into executing the malicious event call.

## Remediation

Always include a CSRF token in sensitive event calls, such as `SingleTransact`.

## Secure example

```
event SingleTransact(address owner, uint value, address to, bytes data,
address created) { require(msg.sender == owner || owner(msg.sender)); }
```

Source lines: 72-421

### ► Finding 8.1: Cross-Site Request Forgery (CSRF) (Critical)

#### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
 // first, take the opportunity to check that we're under the daily limit.
 if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
 // yes - just execute the call.
 address created;
 if (_to == 0) {
 created = create(_value, _data);
 } else {
 if (!_to.call.value(_value)(_data)) throw;
 }
 SingleTransact(msg.sender, _value, _to, _data, created);
 } else {
 // determine our operation hash.
 o_hash = sha3(msg.data, block.number);
 // store if it's new
 if (m_txs[o_hash].to == 0 && m_txs[o_hash].value == 0 &&
m_txs[o_hash].data.length == 0) {
 m_txs[o_hash].to = _to;
 m_txs[o_hash].value = _value;
 m_txs[o_hash].data = _data;
 }
 }
}
```

```

 if (!confirm(o_hash)) {
 ConfirmationNeeded(o_hash, msg.sender, _value, _to, _data);
 }
}
}
}

```

### Explanation

The execute function does not verify the validity of the request data. An attacker can submit a malicious request by manipulating the 'msg.data' variable.

### Impact

Unauthorized actions performed on behalf of authenticated users.

### Entry point

/execute(address, uint256, bytes)

### HTTP methods

POST

### Parameters

\_data, \_to, \_value

### Exploitation steps

- Manipulate the request data to execute unintended actions
- Use the attacker-controlled data to bypass authentication checks

### Example payloads

- "http://malicious.com/transfer?\_to=0&\_value=100&\_data="\x00\x01\x02\x03\x04\x05\x06\x07\x08\x09\x10\x11\x12\x13\x14\x15\x16\x17\x18\x19\x20"

### Conditions

The attacker needs to reach the POST /transfer endpoint while authenticated.

### Remediation

Verify the validity of the request data by using CSRF tokens or similar protection mechanisms

### Secure example

```

function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
 require(sha3(msg.data) != sha3(_data));
 // ...}

```

## ► Finding 8.1: Cross-Site Request Forgery (High)

### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) { // <yes> <report> ACCESS_CONTROL
if ((_data.length == 0 && underLimit(_value)) || m_required == 1) {
 // yes - just execute the call.
 a address created;
 floor _to == 0) {
 created = create(_value, _data);
 } else {
 if (!_to.call.value(_value)(_data))
 nthrow;
 }
}
```

### Explanation

The function does not verify the origin of the request before executing it, making it vulnerable to Cross-Site Request Forgery attacks.

### Impact

Unauthorized actions can be performed on behalf of authenticated users.

### Entry point

/execute

### Execution path

```
External Request -> underLimit/required check -> Execution
External Request -> Reveal Origin Check (Missing) -> Execution
```

### HTTP methods

POST

### Parameters

\_to, \_value, \_data

### Exploitation steps

- User sends malicious payload to the target address.
- Malicious payload is executed without origin verification.

### Example payloads

- `\x00\x01\x02`

### Conditions

Target contract has \_required = 1 or \_to is a valid address.

### Remediation

Verify the origin of the request before executing it.

## Secure example

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
 require(_msg.sender == owner);
 require(bytes(_data).length > 0);
 // ...
}
```

## ► Finding 8.2: Cross-Site Request Forgery (Medium)

### Vulnerable code

```
function hasConfirmed(bytes32 _operation, address _owner) external constant
returns (bool) { return _walletLibrary.delegatecall(msg.data); }
```

### Explanation

The function does not verify the origin of the request before making a delegated call to another contract.

### Impact

Unauthorized actions can be performed on behalf of authenticated users if the delegated contract is compromised.

### Entry point

/hasConfirmed

### Execution path

```
External Request -> delegatecall -> External Contract
External Request -> Reveal Origin Check (Missing) -> Delegation
```

### HTTP methods

GET

### Parameters

\_operation, \_owner

### Exploitation steps

- User sends malicious payload to the target contract.
- Malicious payload is delegated without origin verification.

### Example payloads

- `\x00\x01\x02`

### Conditions

Target contract has `_walletLibrary.delegatecall` with invalid `msg.data`.

## Remediation

Verify the origin of the request before making a delegated call to another contract.

## Secure example

```
function hasConfirmed(bytes32 _operation, address _owner) external constant
returns (bool) { require(_msg.sender == owner); return true; }
```

**Source lines:** 207-477

### ► Finding 8.1: Potential CSRF vulnerability (Low)

#### Vulnerable code

```
// (re)sets the daily limit. needs many of the owners to confirm.
define setDailyLimit(uint _newLimit) onlymanyowners(sha3(msg.data)) external {
 m_dailyLimit = _newLimit;
}
```

#### Explanation

The `setDailyLimit` function uses `onlymanyowners` modifier, which seems to protect against reentrancy attacks, but may not prevent CSRF attacks since the same modifier is used in other functions (e.g., `initWallet`, `kill`).

#### Impact

Unauthorized changes to daily spending limits.

#### Entry point

/setDailyLimit

#### Execution path

```
execute -> setDailyLimit
```

#### HTTP methods

POST

#### Parameters

msg.data

#### Exploitation steps

- Send a request with malicious data to the /setDailyLimit endpoint.

#### Example payloads

- malformed or unexpected data

**Conditions**

Authenticated user with malicious data

**Remediation**

Use `onlyowner` modifier instead of `onlymanyowners` for sensitive functions.

**Secure example**

```
define setDailyLimit(uint _newLimit) onlyowner(sha3(msg.data)) external {
 m_dailyLimit = _newLimit;
}
```

## File 9: /work/project/dataset/vulnerable/access-control/parity\_wallet\_bug\_2.sol

Similarity score: 0.596

Source lines: 1-286

### ► Finding 9.1: Insecure Multi-Sig Wallet (Medium)

#### Vulnerable code

```
function execute(address _to, uint _value, bytes _data) external onlyowner
returns (bytes32 o_hash) {
 // ...
}
```

#### Explanation

The `execute` function does not verify the origin of the request before executing a transaction.

#### Impact

An attacker could use this vulnerability to execute malicious transactions on behalf of an authenticated user.

#### Entry point

/execute

#### Execution path

```
GET /execute
_to = request.target
_value = request.body.value
_data = request.body.data
```

#### HTTP methods

POST

#### Parameters

\_to, \_value, \_data

#### Exploitation steps

- An attacker sends a malicious transaction to the `/execute` endpoint with a forged `\_to` parameter.

#### Example payloads

- ```
"_to": "0x1234567890123456789012345678901234567890", "_value": 1, "_data": "Hello World!"
```

Conditions

The attacker must be authenticated and have a valid `\_to` parameter.

Source lines: 96–414

► Finding 9.1: Missing SameSite Cookie (High)

Explanation

The contract does not set a SameSite attribute for its cookies, making them vulnerable to CSRF attacks.

Impact

An attacker could potentially trick users into executing unintended transactions on their behalf.

Entry point

/execute

Execution path

```
GET /execute HTTP/1.1
Host: example.com
Cookie: session_id=1234567890
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Referer: https://example.com/referer
```

HTTP methods

GET, POST

Parameters

_to, _value, _data

Exploitation steps

- The attacker crafts a malicious URL that includes a CSRF token, which is then included in the request.
- The contract verifies the CSRF token and executes the transaction on behalf of the user.

Example payloads

- `https://example.com/referer?_to=0x1234567890&_value=100&_data=Hello%20World`

Conditions

The attacker can reach POST /execute while authenticated.

Remediation

Set the SameSite attribute for all cookies to prevent CSRF attacks.

Secure example

```
contract.setSameSiteAttribute('LAX')
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)